# Modifier Kipu en autonomie

Scores, outils, arbres, médicaments, examen clinique, affichage, publication sur le web et sur iPhone, avec ou sans Xcode.

---

Dépôt : **github.com/Syssans/urg-kipu** · Site : **syssans.github.io/urg-kipu**
Version du guide : novembre 2026 · Écrit pour l'état du code à cette date

# Sommaire


| | | |
|---|---|---|
| 01 | Comment Kipu fonctionne | le circuit d'une modification |
| 02 | Installer ses outils | navigateur seul, ou PC Windows |
| 03 | Le minimum de code à connaître | lire et écrire sans casser |
| 04 | La carte du projet | quel fichier fait quoi |
| 05 | Modifier un score existant | libellés, points, interprétation |
| 06 | Ajouter un nouveau score | modèle complet |
| 07 | Les outils de calcul | onglet « Calcul » |
| 08 | Les arbres décisionnels | questions, feuilles, scores intégrés |
| 09 | Les fiches médicaments | et les liens automatiques |
| 10 | L'examen clinique | tes phrases, tes sections |
| 11 | Modifier l'affichage | couleurs, textes, accueil, icônes, nom |
| 12 | Vérifier avant de publier | build, lint, lire une erreur |
| 13 | Publier | site web et fichier .ipa |
| 14 | Passer par Xcode | si tu as un Mac qui le fait tourner |
| 15 | La partie native iOS | barre d'onglets, Swift |
| 16 | Dépannage | symptôme → solution |
| 17 | Bonnes pratiques et annulation | revenir en arrière |
| 18 | Annexes | modèles à copier, aide-mémoire |


**Comment lire ce guide**

Les chapitres 1 à 4 sont à lire une fois, dans l'ordre. Ensuite, chaque chapitre se lit seul, comme une fiche : tu vas directement à celui qui correspond à ce que tu veux modifier. Les modèles prêts à copier sont regroupés en annexe (chapitre 18).

# 1 Comment Kipu fonctionne

Kipu est un site web. L'app iPhone est ce même site, emballé dans une coque native. Tu modifies donc un seul code, qui sert aux deux.

## Les briques

| Brique | Rôle, en une phrase |
|---|---|
| **React + TypeScript** | Le langage de l'app. Chaque écran est un fichier `.tsx`, chaque score un fichier `.ts`. |
| **Vite** | L'outil qui transforme le code en site prêt à publier ( `npm run build` ) et qui affiche l'app en direct pendant que tu codes ( `npm run dev` ). |
| **Tailwind CSS** | La mise en forme : des mots-clés comme `text-sm` ou `rounded-xl` directement dans le code des écrans. |
| **Capacitor** | La coque qui transforme le site en app iOS (dossier `ios/` ). |
| **GitHub** | Garde le code et son historique complet. Chaque version est sauvegardée, on peut toujours revenir en arrière. |
| **GitHub Actions** | Des robots gratuits qui, à chaque modification, reconstruisent le site et l'app iOS sur des serveurs de GitHub (dont des Mac). |

## Le circuit d'une modification

**1. Tu modifies**
un fichier, dans le navigateur (github.dev) ou sur ton PC.

→

**2. Tu enregistres sur GitHub**
un « commit » sur la branche `main`.

→

**3. GitHub reconstruit**
le site (≈ 1 min) ; l'app iOS à la demande (≈ 2 min).

→

**4. Tu testes**
sur le site, puis sur l'iPhone via Sideloadly.

**Filet de sécurité**

Si ton code contient une erreur, l'étape 3 échoue et **le site en ligne reste sur la version précédente**, qui fonctionne. Une erreur ne peut donc pas casser le site que tu utilises à l'hôpital. Elle bloque juste la mise à jour, jusqu'à ce que tu la corriges (chapitre 12).

## Vocabulaire Git (le strict nécessaire)

| Mot | Ce que ça veut dire |
|---|---|
| **Dépôt** (repository) | Le dossier du projet, sur GitHub : `Syssans/urg-kipu`. |
| **Commit** | Une sauvegarde nommée de tes modifications (« Ajoute le score HEART »). Chaque commit est une version vers laquelle on peut revenir. |

| | |
|---|---|
| **Push** | Envoyer tes commits de ton PC vers GitHub. Inutile si tu travailles directement dans le navigateur. |
| **Pull / Fetch** | Récupérer sur ton PC ce qui a changé sur GitHub. |
| **Branche** | Une version parallèle du code. Le site est construit à partir de `main` : c'est la seule qui compte pour toi. Travaille directement dessus. |

# 2 Installer ses outils

Deux façons de travailler. Commence par la A, qui ne demande rien. Passe à la B quand tu veux voir le résultat avant de publier.

## Méthode A : tout dans le navigateur (github.dev)

Fonctionne sur n'importe quel ordinateur, même celui de l'hôpital si GitHub n'est pas filtré. Rien à installer.

1. Connecte-toi sur **github.com**, ouvre le dépôt **Syssans/urg-kipu**.
2. Appuie sur la touche `.` (point) du clavier. Un éditeur de code (Visual Studio Code, version web) s'ouvre sur **github.dev**. Tu peux aussi remplacer `github.com` par `github.dev` dans l'adresse.
3. À gauche, l'explorateur de fichiers. Ouvre un fichier, modifie-le. Les fichiers modifiés prennent une lettre **M**.
4. Clique sur l'icône **Contrôle de code source** (trois ronds reliés, à gauche, ou `Ctrl` + `Maj` + `G`).
5. Écris un message court qui décrit le changement, puis clique sur **Commit & Push**. Vérifie que la branche affichée en bas à gauche est `main`.
6. Va dans l'onglet **Actions** du dépôt sur github.com : une coche verte au bout d'une minute ou deux veut dire que le site est à jour.

> **Limite de la méthode A**
>
> Tu ne vois pas le résultat avant de publier. Si tu as fait une faute de frappe, tu ne le sauras qu'en voyant une croix rouge dans Actions. Ce n'est pas grave (le site reste sur l'ancienne version), mais pour les grosses modifications, la méthode B est plus confortable.

## Méthode B : sur ton PC Windows

### Installer (une seule fois)

| Logiciel | Où le trouver | À quoi il sert |
|---|---|---|
| **GitHub Desktop** | desktop.github.com | Récupérer le projet et envoyer tes modifications, avec des boutons plutôt que des commandes. |
| **Node.js** (version LTS) | nodejs.org → bouton « LTS » | Faire tourner l'app sur ton PC et vérifier le code. Prends la version LTS (≥ 22). |
| **Visual Studio Code** | code.visualstudio.com | L'éditeur de code. Gratuit. |

Facultatif dans VS Code : l'extension **Tailwind CSS IntelliSense** (onglet Extensions, à gauche), qui propose les classes de mise en forme pendant que tu tapes.

### Récupérer le projet

1. Ouvre GitHub Desktop, connecte-toi avec ton compte GitHub.
2. **File → Clone repository**, choisis `Syssans/urg-kipu`, garde le dossier proposé (par exemple `Documents\GitHub\urg-kipu`), clique sur **Clone**.

3. En haut, vérifie que **Current branch** indique `main`. Sinon, clique dessus et choisis `main`.
4. Clique sur **Open in Visual Studio Code** (ou dans VS Code : Fichier → Ouvrir le dossier).

### Lancer l'app sur ton PC

1. Dans VS Code : menu **Terminal → Nouveau terminal**. Une ligne de commande s'ouvre en bas, déjà placée dans le dossier du projet.
2. La première fois seulement (et après chaque mise à jour de `package.json`) : tape `npm install` puis `Entrée`. Ça télécharge les dépendances (1 à 2 minutes).
3. Tape `npm run dev`. Ouvre l'adresse affichée dans ton navigateur : **http://localhost:5173/urg-kipu/**.
4. Modifie un fichier et enregistre (`Ctrl` + `S`) : la page se met à jour toute seule, en une seconde.
5. Pour arrêter : clique dans le terminal et fais `Ctrl` + `C`.

**Voir l'app sur ton téléphone pendant que tu codes**

Lance `npm run dev -- --host`. Le terminal affiche une seconde adresse, « Network », du type `http://192.168.1.20:5173/urg-kipu/`. Ouvre-la dans Safari sur ton iPhone, connecté au **même Wi-Fi**. Si Windows demande d'autoriser Node.js dans le pare-feu, accepte pour les réseaux privés.

### Envoyer tes modifications

1. Dans GitHub Desktop, la colonne de gauche liste les fichiers modifiés, avec les lignes changées en vert et rouge à droite. Relis-les.
2. En bas à gauche, écris un résumé (« Corrige le libellé du qSOFA »), clique sur **Commit to main**.
3. Clique sur **Push origin** en haut. C'est envoyé.

**Avant de commencer à travailler**

Clique toujours sur **Fetch origin** (puis **Pull origin** s'il apparaît) dans GitHub Desktop. Tu récupères ainsi les changements faits ailleurs, par exemple depuis github.dev, et tu évites les conflits.

# 3 Le minimum de code à connaître

Pour modifier les scores, les arbres ou les médicaments, tu n'as pas besoin de « savoir programmer ». Tu dois juste savoir lire et respecter quelques signes. Voici tout ce qu'il faut.

## Les briques de base

| Ce que tu vois | Ce que c'est |
|---|---|
| `"Glasgow"` | Un **texte**, toujours entre guillemets droits `"`. Les accents sont permis. |
| `15`, `2.5` | Un **nombre**, sans guillemets. La virgule décimale s'écrit avec un **point** : `2.5`. |
| `true` / `false` | Vrai / faux. |
| `{ label: "Oui", value: 1 }` | Un **objet** : une fiche avec des cases nommées (`nom: valeur`), séparées par des virgules, entre accolades. |
| `[ "a", "b", "c" ]` | Une **liste**, entre crochets, éléments séparés par des virgules. |
| `// commentaire` | Une note pour les humains, ignorée par l'ordinateur. Utile pour te laisser des rappels. |
| `(v) ⇒ v.age ?? 0` | Une **fonction** courte : « à partir des valeurs saisies `v`, donne l'âge, ou 0 s'il est vide ». Le `??` veut dire « sinon ». |
| `a ? b : c` | « Si a, alors b, sinon c ». Exemple : `score ⩾ 2 ? "positif" : "négatif"`. |
| `` `${score} / 3` `` | Un texte entre **accents graves** (`AltGr`+`7` puis `Espace`), dans lequel `${…}` insère une valeur. Donne « 2 / 3 ». |

## Comparaisons et calculs

| Signe | Sens | Signe | Sens |
|---|---|---|---|
| `⩵` | est égal à | `≠` | est différent de |
| `⩾` / `⩽` | supérieur / inférieur ou égal | `>` / `<` | strictement supérieur / inférieur |
| `&&` | ET | `\|\|` | OU |
| `+ - * /` | opérations | `Math.round(x)` | arrondi |
| `x.toFixed(1)` | texte avec 1 décimale | `Math.max(a, b)` | le plus grand des deux |

## Les 6 pièges qui causent 90 % des erreurs

1. **Virgule oubliée** entre deux éléments d'une liste ou d'un objet. Une virgule en trop à la fin, en revanche, est autorisée : n'hésite pas à en mettre.
2. **Guillemets typographiques** `« »` ou `“ ”` à la place de `"` autour d'un texte, souvent après un copier-coller depuis Word. À l'*intérieur* d'un texte, ils sont autorisés.
3. **Guillemet droit à l'intérieur d'un texte** : `"le signe "positif""` casse tout. Écris `"le signe « positif »"` ou `"le signe \"positif\""`. L'apostrophe `'` ne pose pas de problème entre `"`.

4. **Accolade ou crochet non refermé**. Dans VS Code, clique juste après une accolade : sa partenaire s'éclaire.
5. **Virgule décimale** dans un nombre : `2,5` se lit comme deux nombres. Écris `2.5`. Dans un *texte* affiché, en revanche, écris « 2,5 ».
6. **Identifiant en double** : deux scores avec le même `id`, ou deux champs avec le même `id` dans un même score.

**VS Code te prévient**

Une erreur apparaît en **soulignement rouge ondulé**. Passe la souris dessus pour lire le problème. Le panneau **Problèmes** (`Ctrl`+`Maj`+`M`) les liste toutes. Ça marche aussi dans github.dev.

**Raccourcis utiles dans VS Code**

- `Ctrl`+`P` : ouvrir un fichier en tapant son nom (« qsofa »).
- `Ctrl`+`Maj`+`F` : chercher un texte dans tout le projet, par exemple une phrase vue dans l'app, pour trouver où elle est écrite.
- `Ctrl`+`Z` : annuler. `Ctrl`+`/` : mettre une ligne en commentaire.
- `Maj`+`Alt`+`F` : remettre en forme proprement le fichier.

# 4 La carte du projet

Quatre-vingt-dix pour cent de tes modifications se feront dans `src/lib/`, qui contient le contenu médical sous forme de fiches. Les écrans sont dans `src/pages/` et `src/components/`.

```
urg-kipu/
├── src/                          ← tout le code de l'app
│   ├── lib/                      ← le CONTENU
│   │   ├── calculators/          scores (un fichier par score) + index.ts (la liste)
│   │   │   └── types.ts          définition de ce qu'est un score, catégories, couleurs
│   │   ├── tools/                outils de l'onglet Calcul + index.ts
│   │   ├── trees/                arbres décisionnels + index.ts
│   │   ├── drugs/                fiches médicaments + index.ts + sources.ts
│   │   ├── exam/                 examen clinique : template.ts (tes phrases), render.ts
│   │   ├── search.ts             recherche + synonymes (ACR = arrêt cardiaque…)
│   │   ├── favorites.ts          favoris (stockés dans le navigateur)
│   │   └── native.ts             lien avec la partie iOS native
│   ├── pages/                    ← les ÉCRANS (Home.tsx = accueil, About.tsx = À propos…)
│   ├── components/               ← les BRIQUES d'écran (barre du bas, en-tête, carte résultat…)
│   ├── App.tsx                   la liste des adresses (routes) de l'app
│   └── index.css                 les couleurs générales et animations
├── public/                       ← images et icônes du site
├── ios/                          ← le projet Xcode (app iPhone)
│   └── App/App/                  code Swift, icônes, Info.plist
├── scripts/gen-icons.mjs         génère les icônes à partir du dessin du « K »
├── .github/workflows/            ← les robots : deploy.yml (site), ios-ipa.yml (app)
├── vite.config.ts                nom de l'app installable (PWA), adresse du site
├── capacitor.config.ts           nom et identifiant de l'app iOS
├── package.json                  dépendances et commandes npm
└── README.md                     présentation du projet
```

## Les adresses de l'app

Chaque écran a une adresse, définie dans `src/App.tsx`. Utile pour les liens dans les arbres (chapitre 8).

| Adresse | Écran |
|---|---|
| `/` | Accueil (recherche, favoris, récents) |
| `/scores` · `/scores/qsofa` | Liste des scores · un score (par son `id`) |
| `/calcul` · `/calcul/imc` | Liste des outils · un outil |
| `/arbres` · `/arbres/dysnatremie` | Liste des arbres · un arbre |
| `/medicaments/clonazepam` | Une fiche médicament (pas de liste) |
| `/examen` | Examen clinique |
| `/patients` · `/patients/<id>` | Fiches patients · une fiche |
| `/favoris` · `/a-propos` | Favoris · À propos |

**Retrouver où un texte est écrit**

Tu vois une phrase dans l'app et tu veux la changer ? `Ctrl` + `Maj` + `F` dans VS Code, colle un morceau de la phrase (sans accents compliqués si possible) : le fichier apparaît. C'est la méthode la plus rapide pour tout.

# 5 Modifier un score existant

Chaque score est un fichier dans `src/lib/calculators/`. Exemple complet avec le qSOFA, le plus simple.

```
import type { Calculator } from "./types";        // ne pas toucher

export const qsofa: Calculator = {
  id: "qsofa",                                    // identifiant unique, sans accent ni espace → adresse /scores/qsofa
  name: "quick SOFA (qSOFA)",                     // nom complet, en haut de la page du score
  shortName: "qSOFA",                             // nom court, dans les listes
  category: "infectiologie",                      // voir le tableau des catégories
  keywords: ["qsofa", "sepsis", "choc septique", "infection"],   // mots qui le font trouver dans la recherche
  summary: "Repérage rapide au chevet d'un patient à risque de sepsis…",  // sous-titre dans les listes
  fields: [                                       // les questions posées
    { type: "boolean", id: "rr",  label: "Fréquence respiratoire ≥ 22/min", points: 1 },
    { type: "boolean", id: "gcs", label: "Altération de la conscience (Glasgow < 15)", points: 1 },
    { type: "boolean", id: "sbp", label: "Pression artérielle systolique ≤ 100 mmHg", points: 1 },
  ],
  compute: (v) ⇒ (v.rr ?? 0) + (v.gcs ?? 0) + (v.sbp ?? 0),   // le calcul du score
  interpret: (score) ⇒ [                          // ce qui s'affiche selon le résultat
    {
      title: score ⩾ 2 ? "qSOFA positif" : "qSOFA négatif",
      level: score ⩾ 2 ? "high" : "low",          // la couleur de la carte résultat
      scoreLabel: `${score} / 3`,
      detail: score ⩾ 2 ? "Risque accru de mortalité…" : "Ne permet pas d'exclure un sepsis…",
    },
  ],
  source: "Singer et al. (Sepsis-3), JAMA 2016.", // affichée sous le résultat
};
```

## Les modifications courantes

| Je veux… | Je change… |
|---|---|
| Reformuler une question | Le `label` du champ. |
| Changer le nombre de points | Le `points` (case à cocher) ou le `value` d'une option (liste de choix). Vérifie ensuite que `compute` et les seuils de `interpret` sont toujours cohérents. |
| Changer un seuil d'interprétation | Les comparaisons dans `interpret` ( `score ⩾ 2` …). |
| Changer le texte du résultat | `title` (titre de la carte) et `detail` (paragraphe dessous). |
| Changer la couleur du résultat | `level` (tableau ci-dessous). |
| Faire trouver le score par un autre mot | Ajoute le mot dans `keywords`. Pour un synonyme valable partout (ex. « IDM » ↔ « infarctus »), ajoute-le dans `SYNONYM_GROUPS` de `src/lib/search.ts`. |
| Changer sa catégorie | `category`. |

| | |
|---|---|
| Retirer un score de l'app | Dans `src/lib/calculators/index.ts`, supprime son nom de la liste `calculators` et sa ligne `import`. Tu peux garder le fichier. |

## Les couleurs de résultat ( `level` )

| Valeur | Couleur | Usage habituel |
|---|---|---|
| `"low"` | vert | Rassurant, normal, risque faible |
| `"moderate"` | jaune ambré | Intermédiaire, à surveiller |
| `"high"` | orange | Élevé, action nécessaire |
| `"critical"` | rouge | Grave, urgence |
| `"info"` | gris | Information neutre, sans jugement de gravité |

## Les catégories ( `category` )

`"neurologie"`, `"cardiovasculaire"`, `"respiratoire"`, `"infectiologie"`, `"traumatologie"`, `"digestif"`, `"biologie"`, `"pediatrie"`, `"geriatrie"` pour les scores ; `"conversion"` et `"formule"` pour les outils de calcul. Le nom affiché et la couleur de chaque catégorie sont dans `CATEGORY_LABELS` et `CATEGORY_COLORS` de `src/lib/calculators/types.ts`.

**Ajouter une nouvelle catégorie**

Dans `types.ts`, il faut l'ajouter à **trois** endroits : la liste `Category` ( `| "urologie"` ), `CATEGORY_LABELS` ( `urologie: "Urologie",` ) et `CATEGORY_COLORS` ( `urologie: { bg: "bg-teal-500/15", text: "text-teal-300" },` ). Si tu en oublies un, VS Code souligne l'erreur en rouge.

## Les trois types de questions ( `fields` )

### 1. Case à cocher : `"boolean"`

Vaut 1 si cochée, 0 sinon. Idéal pour les scores « 1 point par critère ».

```
{ type: "boolean", id: "age65", label: "Âge ≥ 65 ans", points: 1 },
{ type: "boolean", id: "atcdEp", label: "Antécédent de TVP ou d'EP", points: 1.5, hint: "Précision
affichée en petit" },
```

Dans `compute`, une case cochée vaut **1**, pas son nombre de points. Les `points` servent à l'affichage (« +1,5 » à droite). Pour un score pondéré, multiplie : `(v.atcdEp ?? 0) * 1.5`. Voir l'astuce page suivante pour additionner automatiquement.

### 2. Liste de choix : `"select"`

Une seule réponse possible. Chaque option a un texte ( `label` ) et une valeur ( `value` ).

```
{
  type: "select",
  id: "eye",
  label: "Ouverture des yeux (Y)",
  showPoints: true,             // affiche la valeur à droite de chaque option
  options: [
    { label: "Spontanée", value: 4 },
    { label: "À la demande verbale", value: 3 },
    { label: "À la douleur", value: 2 },
    { label: "Absente", value: 1 },
  ],
},
```

- Tant qu'une liste n'a pas de réponse, le résultat ne s'affiche pas et la question est marquée d'un point orange.
- `defaultValue: 0` pré-sélectionne une option. À réserver aux choix de contexte (artériel/veineux, adulte/enfant), jamais à un item qui compte des points.
- Sans `showPoints` et avec 3 options ou moins, la liste s'affiche en boutons compacts côte à côte.

### 3. Nombre : `"number"`

```
{
  type: "number",
  id: "age",
  label: "Âge",
  unit: "ans",
  min: 0, max: 120,          // hors de ces bornes : « valeur improbable », pas de résultat
  step: 1,
  placeholder: "ex. 65",     // texte gris dans le champ vide
  normText: "7,35-7,45",     // normale affichée en petit à côté du libellé (facultatif)
},
```

Un champ nombre est **facultatif** par défaut. Pour l'exiger avant d'afficher un résultat, ajoute son `id` au score : `requiredNumberFieldIds: ["age", "poids"],`.

### Options communes à tous les types

| Option | Effet |
|---|---|

| | |
|---|---|
| `group: "Critères cliniques"` | Regroupe sous un même intertitre toutes les questions qui ont le même `group`. |
| `visibleIf: (v) ⇒ v.mode === 1` | N'affiche la question que si une autre réponse vaut une certaine valeur. Ici : seulement si le champ `mode` vaut 1. |

## Écrire `compute` et `interpret`

### `compute` : calculer le score

Reçoit `v`, l'ensemble des réponses, rangées par `id`. `v.age` est la réponse au champ dont l'`id` est `"age"`. Une réponse vide vaut `undefined`, d'où le `?? 0` systématique.

```
// Somme simple de cases à cocher ou de listes
compute: (v) ⇒ (v.a ?? 0) + (v.b ?? 0) + (v.c ?? 0),

// Somme pondérée
compute: (v) ⇒ (v.fc ?? 0) * 1.5 + (v.atcd ?? 0) * 1.5 + (v.cancer ?? 0),

// Somme automatique de tous les champs, avec les points des cases à cocher
compute: (v) ⇒ monScore.fields.reduce((total, f) ⇒
  total + (f.type ≡ "boolean" ? (v[f.id] ?? 0) * (f.points ?? 1) : (v[f.id] ?? 0)), 0),

// Point attribué selon un nombre saisi
compute: (v) ⇒ ((v.age ?? 0) ⩾ 65 ? 1 : 0) + (v.diabete ?? 0),
```

Dans la troisième formule, remplace `monScore` par le nom de la constante déclarée en haut du fichier (`export const monScore`).

### `interpret` : choisir le message

Reçoit le score calculé (et les réponses `v` si besoin). Renvoie une **liste** de cartes résultat : la première est la carte principale, colorée.

```
interpret: (score) ⇒ {
  if (score ⩾ 4) {
    return [{ title: "Risque élevé", level: "high", scoreLabel: `${score} points`,
              detail: "Conduite à tenir…" }];
  }
  if (score ⩾ 2) {
    return [{ title: "Risque intermédiaire", level: "moderate", scoreLabel: `${score} points` }];
  }
  return [{ title: "Risque faible", level: "low", scoreLabel: `${score} points` }];
},
```

Les seuils se testent du plus haut au plus bas : le premier `if` vrai renvoie son message, les suivants ne sont pas lus.

### Plusieurs cartes

```
return [
  { title: "Risque élevé", level: "high", scoreLabel: `${score} / 10` },
  { title: "À retenir", level: "info", role: "secondary", detail: "Un complément, en carte
violette discrète." },
];
```

| Case du résultat | Rôle |
|---|---|
| `title` (obligatoire) | Titre de la carte. |
| `level` (obligatoire) | Couleur : `low`, `moderate`, `high`, `critical`, `info`. |
| `scoreLabel` | Le chiffre en grand (« 3 / 5 », « 27,4 kg/m² »). |

| | |
|---|---|
| `detail` | Paragraphe explicatif. |
| `role: "secondary"` | Carte complémentaire, plate et violette, sans halo coloré. |

**Les médicaments deviennent cliquables tout seuls**

Si un `detail` contient le nom d'un médicament qui a une fiche (« clonazépam », « Rivotril »), il devient un lien vers sa fiche automatiquement. Rien à faire.

# 6 Ajouter un nouveau score

Trois étapes : créer le fichier, l'inscrire dans la liste, vérifier.

1. **Crée le fichier** dans `src/lib/calculators/`, par exemple `monScore.ts`. Dans VS Code : clic droit sur le dossier → Nouveau fichier. Le plus simple : copie le fichier d'un score qui a la même structure (cases à cocher → `qsofa.ts` ; listes de choix → `gcs.ts` ; nombres → `spesi.ts` ou un outil), puis adapte-le. Ou pars du modèle de l'annexe A.
2. **Donne-lui un nom de constante unique** : la ligne `export const monScore: Calculator = {`. Et un `id` unique, en minuscules, sans accent ni espace (des tirets sont permis) : `id: "mon-score"`.
3. **Inscris-le** dans `src/lib/calculators/index.ts`, à deux endroits :

```
import { monScore } from "./monScore";   // avec les autres imports, en haut

export const calculators: Calculator[] = [
  gcs,
  nihss,
  …
  mmrc,
  monScore,                              // ← ajouté ; l'ordre ici est l'ordre d'affichage
];
```

4. **Vérifie** : avec `npm run dev`, ouvre `/scores/mon-score`, teste les cas limites (score minimal, score maximal, chaque seuil). Puis `npm run build` (chapitre 12).
5. **Publie** (chapitre 13).

**Exactitude médicale**

Le code ne vérifie pas que ton score est juste. Recopie les critères et les seuils depuis la publication de référence, mets-la dans `source`, et teste au moins un cas dont tu connais le résultat. Le dépôt est public : n'y mets jamais de donnée patient, même dans un commentaire ou un exemple.

**Le nouveau score apparaît partout automatiquement**

Une fois inscrit dans `index.ts`, il est dans l'onglet Scores (sous sa catégorie), dans la recherche, peut être mis en favori et intégré dans un arbre. Rien d'autre à brancher.

# 7 Les outils de calcul

Les outils de l'onglet « Calcul » (IMC, clairance, QTc…) sont construits exactement comme les scores. Seuls le dossier et la liste changent.

| | Score | Outil |
|---|---|---|
| Dossier | `src/lib/calculators/` | `src/lib/tools/` |
| Liste à compléter | `calculators/index.ts` | `tools/index.ts` (liste `tools`) |
| Catégorie | neurologie, cardio… | `"conversion"` ou `"formule"` |
| Import du type | `from "./types"` | `from "../calculators/types"` |
| Adresse | `/scores/<id>` | `/calcul/<id>` |

## Exemple commenté : l'IMC ( `src/lib/tools/bmi.ts` )

```ts
import type { Calculator, Level } from "../calculators/types";

export const bmi: Calculator = {
  id: "imc",
  name: "Indice de masse corporelle (IMC)",
  shortName: "IMC",
  category: "formule",
  keywords: ["imc", "bmi", "poids", "taille", "obésité"],
  summary: "Calcule l'indice de masse corporelle à partir du poids et de la taille.",
  fields: [
    { type: "number", id: "poids",  min: 0.5, max: 400, label: "Poids",  unit: "kg", step: 0.1 },
    { type: "number", id: "taille", min: 30,  max: 250, label: "Taille", unit: "cm", step: 1 },
  ],
  requiredNumberFieldIds: ["poids", "taille"],     // pas de résultat tant que les deux sont vides
  compute: (v) => {
    const poids = v.poids ?? 0;
    const tailleM = (v.taille ?? 0) / 100;
    return tailleM > 0 ? poids / (tailleM * tailleM) : 0;   // éviter la division par 0
  },
  interpret: (score) => {
    let level: Level = "low";
    let title = "Corpulence normale";
    if (score < 18.5) { level = "moderate"; title = "Maigreur"; }
    else if (score ⩾ 30) { level = "high"; title = "Obésité"; }
    return [{ title, level, scoreLabel: `${score.toFixed(1)} kg/m²` }];
  },
  source: "Classification de l'IMC, Organisation mondiale de la santé.",
};
```

**Afficher un résultat à la française**

`score.toFixed(1)` donne « 27.4 » avec un point. Pour une virgule : `score.toFixed(1).replace(".", ",")`.

**Champ compact**

Pour un convertisseur à une seule valeur, ajoute `compact: true` au champ nombre : il s'affiche centré, en plus petit.

# 8 Les arbres décisionnels

Un arbre est une liste de cartes (« nœuds ») reliées entre elles. Fichiers dans `src/lib/trees/`, liste dans `trees/index.ts`.

## Les trois sortes de cartes

| Type | Ce que c'est | Cases principales |
|---|---|---|
| `"question"` | Une question avec des réponses, chacune menant à une autre carte. | `title`, `options` : liste de `{ label, next }` |
| `"leaf"` | Une feuille : la conclusion, au bout d'un chemin. | `title`, `items` (liste à puces), `treatment` (liste) |
| `"score"` | Un score de Kipu affiché dans l'arbre ; la suite dépend du résultat. | `calculatorId`, `branches` : liste de `{ label, test, next }` |

Cases facultatives des questions et des feuilles : `subtitle` (précision sous le titre), `detail` (paragraphe), `warning` (encadré d'alerte), `link` ou `links` (boutons vers une autre page de l'app).

## Squelette d'un arbre

```
import type { DecisionTree } from "./types";

export const monArbre: DecisionTree = {
  id: "mon-arbre",                       // adresse /arbres/mon-arbre
  name: "Nom complet de l'arbre",
  shortName: "Nom court",
  summary: "Une phrase de description.",
  keywords: ["mot", "autre mot"],
  rootId: "debut",                       // la carte affichée en premier
  nodes: {
    debut: {
      type: "question",
      id: "debut",                       // identique à la clé juste au-dessus
      title: "Première question ?",
      options: [
        { label: "Oui", next: "feuille-oui" },     // next = id d'une autre carte
        { label: "Non", next: "glasgow" },
      ],
    },
    "feuille-oui": {                     // guillemets obligatoires si la clé contient un tiret
      type: "leaf",
      id: "feuille-oui",
      title: "Conclusion",
      items: ["Premier point", "Deuxième point"],
      treatment: ["Traitement proposé"],
      warning: "Alerte éventuelle.",
      link: { label: "Voir le score qSOFA", to: "/scores/qsofa" },
    },
    glasgow: {
      type: "score",
      id: "glasgow",
      title: "Évaluer la conscience",
      calculatorId: "gcs",               // l'id d'un score ou d'un outil existant
      branches: [                        // testées dans l'ordre, la première vraie gagne
        { label: "Glasgow ≤ 8",  test: (s) ⇒ s ⩽ 8, next: "feuille-grave" },
        { label: "Glasgow > 8",  test: (s) ⇒ s > 8,  next: "feuille-oui" },
      ],
    },
    "feuille-grave": { type: "leaf", id: "feuille-grave", title: "…", items: ["…"] },
  },
  source: "Protocole de service …, date, rédacteur, validateur.",
};
```

Puis, dans `src/lib/trees/index.ts` : `import { monArbre } from "./monArbre";` et ajoute `monArbre` dans `export const trees = [dysnatremia, seizure, monArbre];`.

**Les erreurs typiques des arbres**

- Un `next` qui pointe vers un `id` qui n'existe pas (faute de frappe) : le bouton ne fait rien apparaître. Vérifie chaque `next`.
- La clé de la carte et son `id` doivent être identiques.
- Des branches de score qui ne couvrent pas tous les cas (un trou entre deux seuils).

Pour s'inspirer d'un arbre réel complet : `src/lib/trees/dysnatremia.ts` (questions et feuilles) et `seizure.ts`.

# 9 Les fiches médicaments

Une fiche par fichier dans `src/lib/drugs/`, liste dans `drugs/index.ts`. Une fois inscrit, le nom du médicament devient cliquable partout dans l'app.

```
import type { Drug } from "./types";

export const clonazepam: Drug = {
  id: "clonazepam",                          // adresse /medicaments/clonazepam
  dci: "Clonazépam",
  brands: ["Rivotril"],                      // noms de marque, cherchables et cliquables
  aliases: ["…"],                            // facultatif : autres noms courants (ex. « Aspirine
»)
  class: "Benzodiazépine anticonvulsivante",
  forms: ["Comprimé sécable 2 mg", "Ampoule 1 mg/1 mL"],
  dosage:
    "Première ligne de posologie.\n" +         // \n = nouvelle puce
    "Deuxième ligne.\n" +
    "_Préparation : 1 ampoule + 1 mL de solvant._",   // _entre tirets bas_ = gris italique
  contraindications: ["Insuffisance respiratoire sévère", "Myasthénie"],
  warning: "Encadré d'alerte (facultatif).",
  notes: "Remarque (facultatif).",
  source: "RCP Rivotril® (ANSM) ; …",
};
```

- `dosage` est un seul texte : chaque `\n` crée une nouvelle puce. Les morceaux se collent avec `+` en fin de ligne.
- Le texte entre tirets bas `_…_` s'affiche en gris italique, pour les dilutions et préparations.
- La source commune du protocole de Dijon est dans `src/lib/drugs/sources.ts` (`DIJON_DILUTIONS_SOURCE`).

**Liens automatiques**

Tout texte de score, d'arbre ou de fiche qui contient la DCI, une marque ou un alias d'une fiche est transformé en lien. Si un nom ne devient pas cliquable, vérifie l'orthographe exacte dans `dci`, `brands` ou ajoute la forme écrite dans `aliases`.

# 10 L'examen clinique

Toutes tes phrases et toutes les options sont dans `src/lib/exam/template.ts`. La mise en forme du texte final (constantes, virgules, majuscules, paragraphes) est dans `src/lib/exam/render.ts`.

## Le principe

- Rien n'est dans l'examen tant que la ligne n'est pas cochée **Normal** ou **Anormal**. Retoucher le bouton actif retire la ligne.
- **Normal** écrit ta phrase habituelle ( `normal` ).
- **Anormal** écrit le début de phrase ( `abnormal` ), puis les options choisies dans chaque groupe ( `choices` ), dans l'ordre du fichier, puis la précision libre.
- **Tout normal** (par section) et **Tout l'examen habituel normal** cochent « Normal » sur les lignes non cochées, sauf celles marquées `optional: true` .

## Une constatation, exemple commenté

```
{
  id: "cough",                    // identifiant unique dans la section
  label: "Toux",                  // nom à l'écran (pas dans le texte)
  normal: "pas de toux",          // phrase quand c'est normal
  abnormal: "toux",               // début de la phrase anormale (peut être "")
  choices: [
    { id: "sputum", label: "Expectorations",
      options: ["sans expectoration", "avec expectorations"] },
    { id: "colour", label: "Aspect des expectorations", multi: true,
      options: ["claires", "jaunâtres", "verdâtres", "hémoptoïques"],
      showIf: { choice: "sputum", any: ["avec expectorations"] } },   // n'apparaît que si…
  ],
}
// Toux + « avec expectorations » + « verdâtres » + « hémoptoïques »
// → « toux avec expectorations verdâtres et hémoptoïques »
```

## Les réglages d'un groupe d'options ( `choices` )

| Réglage | Effet |
|---|---|
| `label` | Titre du groupe de boutons à l'écran. |
| `options` | Les boutons, écrits exactement comme ils apparaîtront dans le texte (avec la préposition : « en fosse iliaque droite »). |
| `multi: true` | Plusieurs choix possibles, reliés en « a, b et c ». Sans lui, un seul choix. |
| `before: "à "` / `after: ")"` | Texte ajouté avant / après les choix. Exemple : `before: "("` et `after: ")"` pour mettre les signes entre parenthèses ; `before: "coté à "` pour « coté à 3/5 ». |
| `sep: ", "` | Séparateur avec ce qui précède. Par défaut une espace (« souffle systolique **3/6** ») ; `", "` pour une nouvelle idée (« …, **reproductible à la palpation** »). |

| | |
|---|---|
| `showIf` | Le groupe n'apparaît, et n'est écrit, que si une des options listées est choisie dans un autre groupe. |

## Modifications courantes

| Je veux… | Je fais… |
|---|---|
| Reformuler une phrase normale | Change `normal`. |
| Ajouter une option (ex. une localisation) | Ajoute le texte dans la liste `options` du bon groupe. |
| Ajouter un groupe d'options | Ajoute un bloc `{ id, label, options }` dans `choices`, à la place voulue dans la phrase. |
| Ajouter une constatation | Ajoute un bloc complet dans `findings`. Ajoute `optional: true` si elle ne doit pas être cochée par « Tout normal ». |
| Ajouter une section (ex. cutané, ORL) | Ajoute un bloc `{ id, title, findings: [...] }` dans la liste `sections`. `end: "."` ferme la phrase par un point ; `sameParagraph: true` la colle à la section précédente. |
| Réutiliser une liste | Les listes communes sont déclarées en haut du fichier (`ABDO_REGIONS`, `LUNG_AREAS`, `SIDES`) : modifie-les une fois, ça change partout. |
| Changer la présentation des constantes | Fonction `vitalsLines` de `render.ts`. |
| Changer l'écriture du Glasgow | Fonction `findingText` de `render.ts` : `` `Glasgow ${…} (Y${g.y}V${g.v}M${g.m})` ``. |

## Le code de transfert

Le bouton « Code pour un autre appareil » transforme l'examen coché en un code court (lettres et chiffres). Sur l'autre appareil, « Reprendre depuis un code » le reconstruit. Le fichier est `src/lib/exam/code.ts` ; tu n'as normalement pas à y toucher.

- Le code contient les constantes, les cases cochées, les options choisies et le Glasgow. Il ne contient **pas le texte libre** (précisions, « autre constatation ») : il est listé sous le code, à retaper.
- Une faute de frappe est détectée : le code est refusé plutôt que mal lu.
- Ce n'est pas un chiffrement : n'importe qui ayant Kipu peut relire un code. Il ne contient jamais d'identité, puisque l'examen n'en a pas.

**Les deux appareils doivent avoir la même version**

Le code dépend de la liste exacte des constatations et des options de `template.ts`. Dès que tu ajoutes, retires ou réordonnes une option, les codes créés avec l'ancienne version sont refusés (message « …pas la même version de Kipu »). Après une modification : recharge le site sur les deux appareils. Attention à l'app iPhone installée par Sideloadly : elle ne se met à jour que quand tu la réinstalles. Pour le transfert, le plus simple est d'utiliser le site (ou l'app ajoutée à l'écran d'accueil depuis Safari), qui se met à jour tout seul.

**Majuscules automatiques**

Chaque section commence par une majuscule et les éléments suivants passent en minuscule ; les sigles comme « TRC » ou « ROT » sont laissés tels quels. Tu peux écrire tes phrases avec ou sans majuscule.

**Confidentialité : ce qu'il ne faut pas changer**

La page n'enregistre rien et n'a aucun champ d'identité. Ne lui ajoute pas de sauvegarde (`localStorage`) ni de champ nom, date de naissance ou IPP. C'est ce qui permet de l'utiliser sur un poste partagé sans risque.

## Les fiches patients

Écrans : `src/pages/PatientsList.tsx` (liste) et `src/pages/PatientPage.tsx` (une fiche). Données et mise en texte : `src/lib/patients.ts`.

- **Où sont les données** : uniquement dans le stockage local du navigateur ou de l'app (`localStorage`, clé `kipu:patients`), sur l'appareil. Rien n'est envoyé, rien n'est synchronisé. Elles restent jusqu'à suppression.
- **Ajouter un champ** : ajoute-le dans l'interface `Patient` et dans la fonction `blank` de `patients.ts` (valeur vide `""`), puis un `<TextField>` ou `<TextArea>` dans `PatientPage.tsx` en copiant un champ existant, et sa ligne dans `patientText` pour qu'il apparaisse dans « Copier la fiche ». Les fiches déjà enregistrées reçoivent automatiquement la valeur vide.
- **Changer le délai de rappel** (48 h) : `OLD_AFTER_MS` dans `patients.ts`.

**Ce sont des données nominatives de santé**

Ne change jamais la clé de stockage pour envoyer ces données ailleurs (serveur, synchronisation, GitHub). Kipu ne chiffre pas les fiches : leur protection repose sur le code de verrouillage du téléphone. Sur un ordinateur partagé, les fiches resteraient lisibles par les utilisateurs suivants.

**Ne pas perdre ses fiches**

Dans Safari, un site non ajouté à l'écran d'accueil peut voir ses données effacées après plusieurs jours sans visite. Utilise Kipu depuis l'icône de l'écran d'accueil ou l'app installée. Supprimer l'app ou effacer les données de Safari efface aussi les fiches.

# 11 Modifier l'affichage

L'apparence se règle avec des « classes » Tailwind : des mots-clés dans `className="…"` des fichiers `.tsx`. Les couleurs générales sont dans `src/index.css`.

## Les couleurs de l'app

Dans `src/index.css`, bloc `@theme` :

```
--color-bg: #0a0a0b;                        // fond noir
--color-surface: rgb(255 255 255 / 0.07);   // fond des cartes (blanc à 7 %)
--color-surface-2: rgb(255 255 255 / 0.12); // cartes appuyées
--color-border: rgb(255 255 255 / 0.12);    // bordures
--color-accent: #e11d2f;                    // rouge
--color-accent-2: #a78bfa;                  // violet Kipu (boutons actifs, liens)
--color-muted: #9a9aa1;                     // texte gris secondaire
```

Changer `--color-accent-2` change le violet **partout** d'un coup. Les codes couleur se choisissent sur n'importe quel « color picker » en ligne (format `#rrggbb`).

**Le violet est aussi écrit ailleurs**

Dans l'app iPhone, la couleur des onglets natifs est dans `ios/App/App/KipuViewController.swift` ( `static let accent = UIColor(red: 0xa7…, green: 0x8b…, blue: 0xfa…)` ), et celle des icônes dans `scripts/gen-icons.mjs` ( `const K = "#a78bfa"` ).

## Aide-mémoire Tailwind

| Classe | Effet |
|---|---|
| `text-xs` `text-sm` `text-base` `text-lg` `text-2xl` | Taille du texte, du plus petit au plus grand |
| `font-medium` `font-semibold` `font-bold` | Graisse du texte |
| `text-white` `text-muted` `text-accent-2` `text-red-400` | Couleur du texte |
| `bg-surface` `bg-accent-2/15` | Couleur de fond ( `/15` = opacité 15 %) |
| `p-4` `px-4` `py-3` `pt-2` | Marge intérieure (tout, horizontale, verticale, en haut). 1 unité = 0,25 rem ≈ 4 px |
| `m-2` `mt-4` `gap-3` | Marge extérieure ; espace entre éléments |
| `rounded-lg` `rounded-xl` `rounded-2xl` `rounded-full` | Coins arrondis |
| `border` `border-border` `border-dashed` | Bordure, sa couleur, en pointillés |

| | |
|---|---|
| `flex` `flex-col` `items-center` `justify-between` | Disposition : en ligne, en colonne, centré, écarté |
| `grid grid-cols-3` | Grille de 3 colonnes |
| `hidden` | Masquer |

Couleurs disponibles : `slate`, `red`, `orange`, `amber`, `yellow`, `lime`, `green`, `emerald`, `teal`, `cyan`, `sky`, `blue`, `indigo`, `violet`, `purple`, `pink`, `rose`, chacune en nuances de `50` (très clair) à `950` (très foncé) : `text-sky-300`, `bg-rose-500/15`.

## Modifications d'affichage courantes

| Je veux… | Fichier et méthode |
|---|---|
| Changer le texte « À propos » | `src/pages/About.tsx` : modifie le texte entre les balises `<p>…</p>`. |
| Changer l'avertissement | `src/components/Disclaimer.tsx` (version longue et version courte « compact »). |
| Modifier l'accueil (tuiles, carte Examen clinique, titres) | `src/pages/Home.tsx`. Chaque tuile est un bloc `<Link to="/scores" …>…</Link>` : copie un bloc complet pour en ajouter une, en changeant `to` et le texte. |
| Changer les catégories de couleur des scores | `CATEGORY_COLORS` dans `src/lib/calculators/types.ts`. |
| Changer les couleurs des résultats | `LEVEL_STYLES` dans le même fichier. |
| Changer le titre d'une page | La ligne `<Header title="…" />` de la page concernée. |
| Changer le nom de l'app installée (PWA) | `vite.config.ts` : `name` et `short_name`. |
| Changer le nom sous l'icône iPhone | `capacitor.config.ts` ( `appName` ) **et** `ios/App/App/Info.plist` ( `CFBundleDisplayName` ). |
| Changer le titre de l'onglet du navigateur | `index.html`, balise `<title>`. |

## Les règles du code d'écran (JSX)

Les fichiers `.tsx` mélangent du code et des balises ressemblant à du HTML :

```
<p className="text-sm text-muted">
  Kipu est développé par <span className="text-slate-200">Anton Millien</span>.
</p>
```

- Chaque balise ouverte `<p>` se referme `</p>`. Une balise sans contenu se ferme elle-même : `<Header title="Examen" />`.
- On écrit `className`, pas `class`.
- Pour écrire les signes `{` `}` `<` `>` dans un texte affiché, entoure-les : `{"< 3 secondes"}`.
- Modifier du texte et des classes est sans risque. Pour ajouter des comportements (boutons qui font quelque chose), inspire-toi d'un écran existant.

## Les icônes de l'app

1. Le dessin du « K » est dans `scripts/gen-icons.mjs` (couleurs `BG` et `K`, tracés `<path>`).
2. Après modification, lance dans le terminal : `node scripts/gen-icons.mjs`. Les images de `public/icons/` sont régénérées.
3. L'icône de l'app iPhone est une image séparée : `ios/App/App/Assets.xcassets/AppIcon.appiconset/AppIcon-512@2x.png` (1024 × 1024 px, sans transparence). Remplace-la par une image de même nom et de même taille.

# 12 Vérifier avant de publier

Deux commandes à lancer dans le terminal avant chaque envoi (méthode B). Elles font exactement ce que fera GitHub : si elles passent chez toi, elles passeront chez GitHub.

| Commande | Ce qu'elle vérifie |
|---|---|
| `npm run build` | Que tout le code est valide (TypeScript) et que le site se construit. **C'est la vérification essentielle.** Termine par « ✓ built in … » si tout va bien. |
| `npm run lint` | Les maladresses de code (variable inutilisée…). Moins grave : un avertissement n'empêche pas la publication. |

## Lire un message d'erreur

```
src/lib/calculators/monScore.ts(14,5): error TS1005: ',' expected.
```

- **Le fichier** : `src/lib/calculators/monScore.ts`.
- **La position** : `(14,5)` = ligne 14, 5e caractère. Dans VS Code, `Ctrl`+`G` puis `14` pour y aller.
- **Le problème** : « ',' expected » = une virgule manque. Regarde la ligne 14 et **la ligne juste avant** : l'oubli est souvent à la fin de la ligne précédente.

## Les erreurs les plus fréquentes, traduites

| Message | Traduction et solution |
|---|---|
| `',' expected` | Virgule manquante entre deux éléments. |
| `'}' expected` / `']' expected` | Accolade ou crochet non refermé, souvent plus haut que la ligne indiquée. |
| `Unterminated string literal` | Un texte dont le guillemet de fin manque, ou un guillemet droit `"` à l'intérieur du texte. |
| `Invalid character` | Un caractère interdit hors d'un texte : souvent des guillemets typographiques `“ ”` collés depuis Word. |
| `Cannot find name 'monScore'` | Le nom est utilisé mais pas importé (ligne `import` manquante dans `index.ts`) ou mal orthographié. |
| `Type '"neuro"' is not assignable to type 'Category'` | Valeur non autorisée : ici la catégorie s'écrit `"neurologie"`. Le message liste souvent les valeurs possibles. |
| `Property 'source' is missing` | Une case obligatoire manque (ici `source`). |
| `Object literal may only specify known properties` | Une case au nom inconnu, souvent une faute de frappe (`lable` au lieu de `label`). |
| `Cannot find module './monscore'` | Le nom de fichier dans l'`import` ne correspond pas exactement (majuscules comprises) au fichier réel. |

## Si tu travailles dans le navigateur (méthode A)

1. Après ton commit, ouvre l'onglet **Actions** du dépôt.
2. Une croix rouge sur « Deploy to GitHub Pages » ? Clique dessus, puis sur le job **build**, puis sur l'étape **Run npm run build** qui est rouge.
3. Le message d'erreur est au même format que ci-dessus. Corrige le fichier dans github.dev, refais un commit : le site se reconstruit.

# 13 Publier

Publier, c'est envoyer un commit sur `main`. Tout le reste est automatique.

## Le site web

1. Envoie ton commit sur `main` (Commit & Push dans github.dev, ou Commit + Push origin dans GitHub Desktop).
2. Onglet **Actions** : « Deploy to GitHub Pages » tourne (rond jaune), puis coche verte au bout d'une à deux minutes.
3. Ouvre **syssans.github.io/urg-kipu**. L'app garde une copie hors ligne : si tu vois l'ancienne version, **recharge la page une ou deux fois**. Sur l'app installée sur l'écran d'accueil, ferme-la complètement et rouvre-la.

## L'app iPhone (sans Mac)

L'app iOS n'est compilée que quand tu le demandes, sur un Mac de GitHub (gratuit tant que le dépôt est public). Lance-la après les modifications que tu veux tester sur l'iPhone.

1. Onglet **Actions** → **Build iOS IPA** → bouton **Run workflow** (branche `main`) → **Run workflow**. Environ deux minutes plus tard, ouvre ce run (coche verte) → en bas, section **Artifacts** → télécharge `Kipu-ipa-N`. Dézippe : tu obtiens `Kipu.ipa`.
2. Sur Windows, ouvre **Sideloadly** (installé avec iTunes et iCloud *téléchargés depuis le site d'Apple*, pas depuis le Microsoft Store).
3. Branche l'iPhone en USB, glisse `Kipu.ipa` dans Sideloadly, entre ton identifiant Apple, clique sur **Start**.
4. Première fois seulement, sur l'iPhone : Réglages → Confidentialité et sécurité → **Mode développeur** (redémarrage), puis Réglages → Général → **VPN et gestion de l'appareil** → faire confiance à ton identifiant Apple.

**La limite des 7 jours**

Avec un identifiant Apple gratuit, l'app installée par Sideloadly cesse de s'ouvrir au bout de 7 jours. Il suffit de la réinstaller de la même façon (les favoris sont conservés). Avec un compte développeur Apple payant (99 €/an), c'est un an.

**Les fichiers .ipa sont gardés 30 jours**

Passé ce délai, l'artefact disparaît de GitHub. Relance simplement **Run workflow** sur « Build iOS IPA » pour en produire un nouveau.

# 14 Passer par Xcode

Xcode n'est pas indispensable : GitHub Actions compile l'app à ta place. Mais si tu as accès à un Mac qui le fait tourner, il permet d'installer directement sur l'iPhone et de déboguer.

## Prérequis

- Un Mac avec une version de macOS récente, compatible avec **Xcode 26** (nécessaire pour l'apparence iOS 26 de l'app).
- De la place : Xcode et le kit iOS occupent plusieurs dizaines de Go. Les simulateurs supplémentaires sont facultatifs.
- Xcode (gratuit, sur le Mac App Store), Node.js LTS, et le projet cloné (GitHub Desktop existe aussi sur Mac).
- Un identifiant Apple (gratuit) et l'iPhone avec son câble.

## Première installation sur l'iPhone

1. Dans le Terminal du Mac, place-toi dans le dossier du projet, puis : `npm install` et `npm run ios`. Cette commande construit l'app web pour iOS, la copie dans le projet Xcode, et ouvre Xcode.
2. Dans Xcode, la colonne de gauche montre le projet **App**. Clique sur **App** tout en haut, puis sur la cible **App**, onglet **Signing & Capabilities**.
3. **Team** → **Add an Account…** → connecte ton identifiant Apple → choisis « (Personal Team) ». Laisse « Automatically manage signing » coché.
4. Si Xcode affiche « Failed to register bundle identifier », change **Bundle Identifier** pour quelque chose d'unique, par exemple `fr.kipu.urgences.anton`.
5. Branche l'iPhone, déverrouille-le, accepte « Faire confiance ». En haut de la fenêtre Xcode, choisis ton iPhone comme destination.
6. Clique sur ▶ (Run) ou `⌘`+`R`. Première fois : active le Mode développeur et fais confiance au profil sur l'iPhone (comme au chapitre 13), puis relance ▶.

## Après chaque modification du code web

Xcode ne voit pas tout seul les changements de `src/`. Dans le Terminal :

```
npm run build:native     # construit l'app web en version iOS (dossier dist-native)
npx cap sync ios         # la copie dans le projet Xcode
```

Puis ▶ dans Xcode. ( `npm run ios` fait les deux et rouvre Xcode.)

**Tester plus vite**

Pour des modifications de contenu (scores, textes), le site en local ( `npm run dev` ) ou sur l'iPhone via `--host` suffit. Passe par Xcode seulement pour vérifier le rendu dans l'app native ou modifier la partie Swift.

## Déboguer avec Safari

1. Sur l'iPhone : Réglages → Apps → Safari → Avancé → **Inspecteur web** activé.
2. Sur le Mac : Safari → Réglages → Avancé → cocher « Afficher les fonctionnalités pour les développeurs web ».
3. App lancée depuis Xcode, dans Safari du Mac : menu **Développement** → ton iPhone → Kipu. Tu vois la console (erreurs en rouge) et peux inspecter chaque élément de la page.

L'inspection n'est possible qu'avec une app lancée depuis Xcode en mode Debug. L'.ipa de GitHub Actions est compilée en mode Release.

# 15 La partie native iOS

Dans l'app iPhone, la barre de titre et la barre d'onglets du bas sont de vrais éléments iOS, dessinés en Swift. Sur le site, ce sont des éléments web. Les deux doivent rester synchronisés.

## Les fichiers

| Fichier | Rôle |
|---|---|
| `ios/App/App/KipuViewController.swift` | Barre de titre, bouton retour, barre d'onglets, couleur violette native. |
| `src/lib/native.ts` | Le pont entre le code web et le Swift. |
| `src/components/BottomNav.tsx` | Barre d'onglets du site, et synchronisation avec les onglets natifs. |
| `src/components/Header.tsx` | Barre de titre web, ou envoi du titre à la barre native. |
| `ios/App/App/Assets.xcassets/tab-*.imageset` | Icônes des onglets natifs (PNG 50 × 50 et 75 × 75 px, forme sur fond transparent : iOS applique lui-même la couleur). |
| `ios/App/App/Info.plist` | Nom affiché, orientations, réglages système. |

## Modifier les onglets du bas

Les onglets sont décrits **deux fois**, dans le même ordre, et doivent correspondre :

```
// src/components/BottomNav.tsx
const items = [
  { to: "/", label: "Accueil", icon: HomeIcon, end: true },
  { to: "/scores", label: "Scores", icon: CalcIcon },
  …
];

// ios/App/App/KipuViewController.swift
private let tabs: [(title: String, icon: String)] = [
    ("Accueil", "tab-home"),
    ("Scores", "tab-scores"),
    …
]
```

- **Renommer un onglet** : change le `label` dans BottomNav.tsx et le titre dans le Swift.
- **Changer l'ordre** : déplace la ligne dans les deux listes. L'iPhone repère l'onglet touché par sa position : si l'ordre diffère, un appui ouvre le mauvais écran.
- **Ajouter un onglet** : une ligne dans chaque liste, une icône web (fonction `…Icon` en bas de BottomNav.tsx) et un nouveau dossier `tab-xxx.imageset` (copie un dossier existant, remplace les deux PNG, et corrige les noms dans son `Contents.json`). iOS affiche bien 5 onglets ; au-delà, prévois plutôt une tuile sur l'accueil.

**Modifier du Swift sans Mac**

C'est possible : modifie le fichier dans github.dev ou VS Code, publie, lance « Build iOS IPA » (Run workflow) et regarde son résultat. En cas d'erreur Swift, la croix rouge apparaît à l'étape **Build App.app (unsigned)** ; le message indique le fichier, la ligne et le problème, comme pour TypeScript. Reste sur de petites modifications (textes, couleurs, ordre).

# 16 Dépannage

| Symptôme | Cause probable | Solution |
|---|---|---|
| « npm n'est pas reconnu » | Node.js pas installé, ou terminal ouvert avant l'installation. | Installe Node.js LTS, puis ferme et rouvre VS Code. |
| « L'exécution de scripts est désactivée sur ce système » | Sécurité de PowerShell sous Windows. | Dans le terminal VS Code, choisis le profil **Command Prompt** (flèche à côté du +), ou tape une fois : `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` et confirme si une question s'affiche. |
| Page blanche en local | Mauvaise adresse, ou erreur dans le code. | L'adresse doit finir par `/urg-kipu/`. Sinon, regarde le terminal et la console du navigateur (`F12` → Console). |
| « Port 5173 is in use » | Un autre `npm run dev` tourne déjà. | Ferme l'autre terminal, ou utilise l'adresse avec le nouveau port affiché. |
| Croix rouge dans Actions | Erreur dans le code. | Lis l'erreur (chapitre 12), corrige, refais un commit. Le site en ligne n'est pas touché en attendant. |
| Le site n'a pas changé après publication | Copie hors ligne de l'app, ou déploiement pas fini. | Vérifie la coche verte dans Actions, puis recharge deux fois. Sur l'app installée : ferme-la complètement. |
| Mon nouveau score n'apparaît pas | Pas inscrit dans `index.ts`. | Ajoute l'`import` et le nom dans la liste (chapitre 6). |
| Le résultat ne s'affiche jamais | Une liste de choix sans réponse, un nombre obligatoire vide ou hors bornes. | Repère le point orange ou la mention « valeur improbable ». Vérifie `min` / `max` et `requiredNumberFieldIds`. |
| Le score calculé est faux | Oubli dans `compute`, ou `id` mal recopié (`v.ag` au lieu de `v.age`). | Chaque `v.xxx` doit correspondre exactement à un `id` de champ. |
| Un bouton d'arbre ne fait rien apparaître | `next` vers un `id` inexistant. | Corrige l'orthographe du `next`. |
| GitHub Desktop : « conflict » | Le même fichier a été modifié sur GitHub et sur ton PC. | Ouvre le fichier dans VS Code : il propose « Accept Current / Incoming / Both ». Garde la bonne version, enregistre, puis commit. Pour l'éviter : **Fetch** avant de commencer. |
| Sideloadly ne voit pas l'iPhone | iTunes du Microsoft Store, ou iPhone verrouillé. | Installe iTunes et iCloud depuis le site d'Apple ; déverrouille l'iPhone et accepte « Faire confiance ». |
| L'app iPhone ne s'ouvre plus | Les 7 jours de signature gratuite sont passés. | Réinstalle l'.ipa avec Sideloadly. |

| | | |
|---|---|---|
| « Build iOS IPA » rouge, étape xcodebuild | Erreur Swift, ou changement côté GitHub/Apple. | Lis la première ligne contenant « error: ». Si ton dernier commit ne touche pas à `ios/`, relance une fois avec **Re-run jobs**. |
| « Build iOS IPA » ou le site restent « Queued » longtemps | Incident chez GitHub. | Patiente ; l'état des services est sur githubstatus.com. |

# 17 Bonnes pratiques et annulation

## Les bonnes habitudes

- **Une modification à la fois**, un commit par modification, avec un message clair (« Ajoute le score HEART », « Corrige le seuil du CRB-65 »). En cas de problème, tu sauras lequel annuler.
- **Copier un fichier qui marche** plutôt que partir de zéro.
- **Tester les cas limites** d'un score : minimum, maximum, et chaque seuil exactement (score = 2 quand le seuil est `⩾ 2`).
- **Toujours une source** dans `source`, avec l'année. Pour un protocole local : établissement, date, rédacteur, validateur.
- **Jamais de donnée patient** dans le dépôt : il est public, et tout l'historique reste consultable même après suppression.
- **Mettre à jour le README** quand tu ajoutes un contenu important : c'est la mémoire du projet.

## Revenir en arrière

| Situation | Solution |
|---|---|
| Pas encore de commit, je veux tout annuler sur un fichier | GitHub Desktop : clic droit sur le fichier → **Discard changes**. Dans github.dev ou VS Code : icône ↶ à côté du fichier dans « Contrôle de code source ». |
| Commit fait mais pas encore envoyé (Push) | GitHub Desktop : bouton **Undo** en bas à gauche, juste après le commit. Les modifications reviennent, non enregistrées. |
| Commit déjà publié, il a cassé quelque chose | GitHub Desktop : **History** → clic droit sur le commit → **Revert changes in commit**, puis **Push origin**. Un nouveau commit annule l'ancien ; l'historique est conservé. |
| Retrouver l'ancienne version d'un fichier | Sur github.com, ouvre le fichier → bouton **History** : chaque version est consultable et copiable. |

> **Rien n'est jamais perdu**
> Tant qu'une version a été envoyée sur GitHub, elle est récupérable. N'aie pas peur d'essayer : au pire, un « Revert » remet tout comme avant.

## Mettre à jour les dépendances (rarement)

Les bibliothèques (React, Vite, Capacitor…) sortent régulièrement de nouvelles versions. Tant que tout fonctionne, **ne touche à rien** : une mise à jour majeure peut demander des adaptations. Si tu dois le faire, `npm outdated` les liste ; les versions sont dans `package.json`.

# 18 Annexes

## A. Modèle de score à cases à cocher

```
import type { Calculator } from "./types";

export const monScore: Calculator = {
  id: "mon-score",
  name: "Nom complet du score",
  shortName: "Nom court",
  category: "cardiovasculaire",
  keywords: ["mot-clé", "autre mot-clé"],
  summary: "À quoi sert ce score, en une phrase.",
  fields: [
    { type: "boolean", id: "critere1", label: "Premier critère", points: 1 },
    { type: "boolean", id: "critere2", label: "Deuxième critère", points: 1 },
    { type: "boolean", id: "critere3", label: "Troisième critère", points: 2 },
  ],
  compute: (v) => (v.critere1 ?? 0) * 1 + (v.critere2 ?? 0) * 1 + (v.critere3 ?? 0) * 2,
  interpret: (score) => {
    if (score >= 3) {
      return [{ title: "Risque élevé", level: "high", scoreLabel: `${score} / 4`, detail:
"Conduite à tenir." }];
    }
    if (score >= 1) {
      return [{ title: "Risque intermédiaire", level: "moderate", scoreLabel: `${score} / 4` }];
    }
    return [{ title: "Risque faible", level: "low", scoreLabel: `${score} / 4` }];
  },
  source: "Auteur et al., Revue, année.",
};
```

## B. Modèle de score à listes de choix

```
import type { Calculator } from "./types";

export const monScoreListes: Calculator = {
  id: "mon-score-listes",
  name: "Nom complet",
  shortName: "Nom court",
  category: "neurologie",
  keywords: ["mot-clé"],
  summary: "Description.",
  fields: [
    {
      type: "select", id: "item1", label: "Premier item", showPoints: true,
      options: [
        { label: "Normal", value: 0 },
        { label: "Modéré", value: 1 },
        { label: "Sévère", value: 2 },
      ],
    },
    {
      type: "select", id: "item2", label: "Deuxième item", showPoints: true,
      options: [
        { label: "Absent", value: 0 },
```

```
        { label: "Présent", value: 3 },
      ],
    },
  ],
  compute: (v) => (v.item1 ?? 0) + (v.item2 ?? 0),
  interpret: (score) => [
    { title: score >= 3 ? "Positif" : "Négatif", level: score >= 3 ? "high" : "low", scoreLabel:
`${score} / 5` },
  ],
  source: "Référence.",
};
```

## C. Modèle d'outil de calcul (onglet Calcul)

```
import type { Calculator } from "../calculators/types";

export const monOutil: Calculator = {
  id: "mon-outil",
  name: "Nom complet de l'outil",
  shortName: "Nom court",
  category: "formule",                  // ou "conversion"
  keywords: ["mot-clé"],
  summary: "Ce que calcule l'outil.",
  fields: [
    { type: "number", id: "a", label: "Première valeur", unit: "mmol/L", min: 0, max: 200, step:
0.1 },
    { type: "number", id: "b", label: "Deuxième valeur", unit: "g/L",    min: 0, max: 100, step:
0.1 },
  ],
  requiredNumberFieldIds: ["a", "b"],
  compute: (v) ⇒ (v.a ?? 0) + 0.5 * (v.b ?? 0),
  interpret: (score) ⇒ [
    { title: "Résultat", level: "info", scoreLabel: `${score.toFixed(1).replace(".", ",")} mmol/L`
},
  ],
  source: "Référence de la formule.",
};
```

À inscrire dans `src/lib/tools/index.ts` : `import { monOutil } from "./monOutil";` et ajouter `monOutil` dans la liste `tools`.

## D. Modèle de fiche médicament

```
import type { Drug } from "./types";

export const monMedicament: Drug = {
  id: "mon-medicament",
  dci: "DCI",
  brands: ["Marque"],
  class: "Classe thérapeutique",
  forms: ["Forme 1", "Forme 2"],
  dosage:
    "Indication 1 : posologie.\n" +
    "Indication 2 : posologie.\n" +
    "_Préparation : …_",
  contraindications: ["Contre-indication 1", "Contre-indication 2"],
  source: "RCP (ANSM) ; protocole …",
};
```

À inscrire dans `src/lib/drugs/index.ts` (import + liste `drugs`).

## E. Aide-mémoire des commandes

| Commande (terminal) | Effet |
|---|---|
| `npm install` | Installe les dépendances (première fois, ou après changement de `package.json`). |
| `npm run dev` | Lance l'app en local sur `http://localhost:5173/urg-kipu/`, avec mise à jour en direct. |
| `npm run dev -- --host` | Idem, accessible depuis le téléphone sur le même Wi-Fi. |
| `npm run build` | Vérifie tout le code et construit le site (dossier `dist/`). |
| `npm run preview` | Affiche le site construit par `build`, comme en ligne. |
| `npm run lint` | Repère les maladresses de code. |
| `node scripts/gen-icons.mjs` | Régénère les icônes du site. |
| `npm run build:native` | Construit la version pour l'app iOS (dossier `dist-native/`). |
| `npx cap sync ios` | Copie cette version dans le projet Xcode. |
| `npm run ios` | Les deux précédentes, puis ouvre Xcode (Mac uniquement). |
| `Ctrl` + `C` | Arrête la commande en cours dans le terminal. |
| `↑` | Rappelle la commande précédente. |

## F. Check-list avant chaque publication

1. J'ai récupéré la dernière version (**Fetch origin**) avant de commencer.
2. Ma modification fonctionne dans `npm run dev`, cas limites compris.
3. `npm run build` se termine par « ✓ built ».
4. J'ai relu les lignes modifiées dans GitHub Desktop (vert/rouge).
5. Aucune donnée patient, aucune information personnelle dans le code.
6. La source est renseignée pour tout contenu médical nouveau ou modifié.
7. Message de commit clair, envoi sur `main`.
8. Coche verte dans Actions, puis vérification sur le site (recharger deux fois).

## G. Liens utiles

| Quoi | Où |
|---|---|
| Le dépôt | github.com/Syssans/urg-kipu |
| Éditer dans le navigateur | github.dev/Syssans/urg-kipu |
| Le site | syssans.github.io/urg-kipu |
| Les robots (builds, .ipa) | github.com/Syssans/urg-kipu/actions |
| Documentation Tailwind (classes) | tailwindcss.com/docs |

| | |
|---|---|
| Documentation Capacitor (iOS) | capacitorjs.com/docs |
| Sideloadly | sideloadly.io |
| État des services GitHub | githubstatus.com |